Software Development for Robotics
Assignment manual (v2025.1)

Jan Broenink, Gijs van Oort, Islam Khalil

Tuesday 13th January, 2026

Contents

0 Introduction **1**
0.1 Context . . . 1
0.2 Organisation . . . 1
0.3 Equipment / Textbook . . . 1
0.4 Assignments . . . 2
0.5 Planning . . . 2
0.6 Submitting your work . . . 2
0.7 Responsibility and fraud . . . 2
0.8 Quality of the results . . . 4
0.9 Document History and Disclaimer . . . 4

1 First Assignment **5**
1.1 File structure and build process . . . 5
1.2 BMI Calculator . . . 6
1.3 *Classical only*: Heart Rates . . . 7
1.4 Questions . . . 7
1.5 Submission . . . 7

2 The Maze **9**
2.1 Arrays . . . 9
2.2 Recursion . . . 9
2.3 Questions . . . 10
2.4 Submission . . . 10

3 Lists and searching **11**
3.1 Linked List . . . 11
3.2 *Classical only*: Tree . . . 11
3.3 Questions . . . 12
3.4 Submission . . . 12

4 Inheritance **13**
4.1 Package Inheritance Hierarchy . . . 13
4.2 *Classical only*: Customer hierarchy . . . 13
4.3 Questions . . . 14
4.4 Submission . . . 14

5 Sequence controller **16**
5.1 RELbot controller architecture . . . 16

5.2 Getting familiar with ROS2 . . . 17
5.3 Sequence Controller . . . 17
5.4 Questions . . . 18
5.5 Submission . . . 18

6 Final Assignment SDfR: Controlling the RELbot 19
6.0 Introduction . . . 19
6.1 Getting more familiar with ROS2 . . . 21
6.2 Detect a coloured object . . . 21
6.3 Sequence Controller . . . 22
6.4 Controlling the physical RELbot . . . 23
6.5 Final Integration . . . 24
6.6 Extra Functionality (Bonus Assignment) . . . 25
6.7 Submission . . . 25

A Change Log 27

B Course-Specific Framing for PPD Projects 28
B.1 Describe the CPS architecture of your robotic system – both SDfR and ASDfR . . . 28
B.2 Specifics for ASDfR . . . 28
B.3 Combined SDfR and ASDfR . . . 29
B.4 Hints on producing diagrams . . . 29

C Useful resources 30
C.1 Books . . . 30
C.2 Online . . . 30

Bibliography 31

0 Introduction

0.1 Context

The goal of this practical work is to train yourself in software development using C++, focusing on the essentials of the language and learning good programming practices. This implies that structuring and documenting the delivered software is as important as its performance, that is computing the correct answers reasonably fast.

To reach this goal, the assignments start as training assignments and end with a somewhat larger project.

A second goal is to understand and use ROS2, the Robotic Operating System, software to combine parts (software components) to produce software for robots.

It provides operating-system like services, but it is *not* an operating system itself. It needs linux to run[1]. We use the RELbot, the education mobile robot of the Robotics Education Lab of MSc Robotics.

You need to have a C++ development environment installed on your *own* laptop.

For installing necessary tools on your laptop, software development using C++ and C++ projects, and software development for and use of a RELbot, see the *RELbot Software and Usage Manual (v2025.n)* by van Oort et al. (2025), in this assignment manual referred to as the *RELbot Manual*.

Some tips and links for further reading on C++ development are in Appendix C.

0.2 Organisation

The practical assignments consist of 5x8 + 1x12 + 3x4 hours as practicals where TAs are present. During this time each group (2 persons) must complete the practical assignments. At the end of each practicals week (the first 4, 5th week more time) you must submit an assignment via the corresponding Assignment in Canvas. Each assignment is described in a separate chapter in this document. Chapter 6 describes the final project, and takes about half of the course. See Section 0.5 for exact dates.

You can ask questions regarding the assignments to the Teaching Assistants, either face2face when you are present at the practicals, or online when you are at home during the practicals. Asking questions at other times is also possible. See Canvas about what communication forms we offer (page *Organisation of Software Development for Robotics*).

0.2.1 PPD version versus Classical version of the course

MSc-Robotics students who take this class as compulsory course must do the PPD[2] variant of this course. This means, they can skip certain sub-assignments (those are indicated), and contribute to their PPD case using material from this course. Next to the *framing* section of the PPD manual, more information about SDfR and PPD is in Appendix B.

Those who do *not* need to do the PPD version, do the *Classical* version of this course, that is all the assignments in this document.

0.3 Equipment / Textbook

You have to use your 'own' computer / laptop.

[1] Well, in theory you can use Windows, but it works significantly better on Linux.
[2] Professional and Personal Development, formerly named Challenge-Based Learning.

Next to this assignment manual, we use the textbook by P. J. Deitel and H. M. Deitel (2017). The assignments in this assignment manual are based on the assignments in this textbook.

You need to have a C++ development environment installed on your *own* laptop. See the separate RELbot manual (van Oort et al., 2025) for more detailed information on what to use and how to install it.

0.4 Assignments

This document contains multiple assignments, growing in creativity/freedom and complexity. The subjects of the assignments are based on the content of the lectures.

Next to the software-development parts, the assignments also have questions to be answered as part of each assignment.

0.5 Planning

Week	*Practicals*	*Subject*	*Deitel & Deitel*	*Submission deadline*
7	9 Feb, 11 Feb	1. Installation & Get Familiar	Chap. 1, 2, 3	11 Feb
8	16 Feb, 18 Feb	2. Arrays, Pointers, Recursion	Chap. 6, 7, 8	18 Feb
9	*Spring break*	–	–	
10	2 Mar, 4 Mar	3. Lists & Trees	Chap 19	4 Mar
11	9 Mar, 11 Mar	4. Inheritance	Chap. 9, 11, 12	11 Mar
12	16 Mar, 18 Mar	5. Assignment 5	ROS tutorials	25 Mar
13	23, 25, 26 Mar	6. Final Assignment	-	-
14	see Canvas	6. Final Assignment in Lab	-	-
15	see Canvas	6. Final Assignment in Lab	-	-
16	see Canvas	6. Final Assignment in Lab	-	17 Apr

Submission deadline is the very late evening of the last half day of each practical: usually a Wednesday of that week, effectively Thursday 04:00.

Note the following:

- During the practicals and Labs the TAs are available to answer questions both virtual and face2face. The queueing tool Horus is available to queue up for *all* questions.
- Presence on Campus when the practicals are scheduled on Campus is not necessary. Obviously, being there makes it easier to ask questions and get answers.

0.6 Submitting your work

You have to submit your work (sources etc, design decisions, and answers to questions) via Canvas. Be sure that the TAs can easily compile and run your code on their machines.

Each submission is graded. The grade is computed as a weighted sum of all grades, see Canvas and the lectures for details.

0.7 Responsibility and fraud

All group members share responsibility for the work handed in. When you distribute work among group members, check each other's work, discuss the work among all group members, do you understand it all?

You must submit original work.

Plagiarism, i.e. copying of someone else's work without proper reference, is a serious offence which in all cases will be reported to the Examination Board. Refer to UT's Student Charter Student Charter, 2020.

In cases where a non-trivial amount of work is copied *with* proper reference, indicate which parts are copied and which parts are your own original work. The copied work will not be considered for grading.

This also holds for the assignments where code is explicitly provided (e.g. Assignments 3 and 6): only your extensions are considered.

0.7.1 What constitutes fraud?

Material written by Arend Rensink is reused here.

When it comes down to handing in assignments, every year there are students who do not understand the borderline between, on the one hand, cooperating and discussing solutions between groups (which is allowed), and on the other, copying or sharing solutions (which is forbidden and counted as fraudulent behaviour). Here are some scenarios which may help in making this distinction.

- **Scenario.** Peter and Lisa are quite comfortable with programming and have pretty much finished the assignment. Mark and Wouter, on the other hand, are struggling and ask Lisa how she has solved it. Lisa, a friendly girl, shows her solution and takes them through it line by line. Mark and Wouter think they now understand and go off to create their own solution, based on what they saw. Is this allowed or not?
  Verdict. No problem here, everything is in the green. It is perfectly fine and allowed for Lisa to explain her solution, even very thoroughly. The important point is that in implementing it themselves and testing their own solution, Mark and Wouter are still forced to think about what is happening and will gain the required understanding, though probably they will not get as much out of it as Lisa (explaining stuff to others is about the best possible way to learn it better yourself!).
- **Scenario.** The start is as in the previous case. However, while Mark and Wouter implement their own solution, inspired by that of Lisa, some error crops up which they do not understand. Lisa has left by now; after they mail her, still trying to be helpful she sends them her solution for them to inspect. They inspect it so closely that in the end their solution is indistinguishable from Peter and Lisa's, except for the choice of some variable names and the comments they added themselves. Is this allowed or not?
  Verdict. This is now a case of fraud. Three are at fault: Lisa for enabling fraud by sending her files (even if it was meant as a friendly gesture) and Mark and Wouter for copying the code. Peter was not involved, developed his own solution (together with Lisa) and is innocent.
- **Scenario.** Alexandra and Nahuel are not finished, and the deadline is very close. The same holds for Simon and Jaco. On the Friday night train home, Jaco and Nahuel meet and during the 2-hour train ride work it out together. They type in the same solution and hand it in on behalf of their groups. Is this allowed or not?
  Verdict. This is also a case of fraud. Actually there are two problems here. The first is that both Nahuel and Jaco handed in code on behalf of their groups that had been developed by them alone, without their partners. This is unwise and against the spirit of the assignment (Alexandra and Simon also need to master this stuff!) but essentially undetectable and not fraudulent. The second problem is that the solution was developed, and shared, in collaboration between two groups; this is definitely forbidden. All four students are culpable; Alexandra and Simon cannot hide behind the fact that they did not partake in the collaboration, as they were apparently happy enough to have their name on the solutions and pretend they worked on it, too.

Note that we are not on a witch-hunt here: let us stress again that cooperating and discussing assignments is OK, even encouraged; it is at the point where you start copying or duplicating pieces of code that you cross the border.

0.8 Quality of the results

The quality of the code will be judged, implying:

- The code must compute the correct values in a reasonable amount of time.
- The structure of the code must be clear, and easily understandable by the TAs.
  You can use *pseudo code* to structure your design, see P. J. Deitel and H. M. Deitel (2017, Sec 4.3 and Sec. 4.11).
- Names of identifiers must follow an appropriate naming scheme.
  Follow the examples as given in P. J. Deitel and H. M. Deitel (2017).
- The code must contain effective comments.
  Follow the style as shown in P. J. Deitel and H. M. Deitel (2017), and explained in their Chapter 2. No comments or irrelevant comments lead to an insufficient mark.

See also tips and hints on the slides dealing with programming style and design decisions.

0.9 Document History and Disclaimer

When you find a mistake or have remarks about this document, please contact one of the TAs or one of the teachers.

As we also continuously improve this document, newer versions appear regularly, see its date and version number. Changes are summarized in the Changelog, in Appendix A. The latest version can be found on Canvas. Be sure to always use the latest version of this guide.

This document has seen many versions over the years. Contributors to this document, next to the teacher, are: Jan Broenink, Gijs van Oort, Luuk Lenders, Ilyas Raoudi, Frank Hooglander, Jelle Hierck, Jurrijn Pott, Wolfgang Baumgartner, Johan Engelen, Sander Grimm, Silke Hofstra, Karim Kok, Frits Kuipers, Laurie Overbeek, Chris Zeinstra, Peter Kingma, Hidde Hoorwerg, Erik vd Ploeg.

Development and documentation of the RELbot is by Gijs van Oort, Sander Smits, Marcel Schwirtz, Marjon Kuipers, Artur Gąsienica, and Jan Broenink. Special thanks to Arend Rensink for the Subsection “What constitutes fraud?”.

1 First Assignment

Use the following *Good Programming Practices* throughout your work:

- Use the standard comment header, see Chapter 5 (Sections 5.3 and 5.4) of the RELbot Manual (van Oort et al., 2025), for every file you create.
- Use the file and folder structure as explained in Chapter 5 (Section 5.2) of the RELbot Manual for your work. This means that you split each subassignment to a different folder.

Note that splitting code into header files (extension `.h` / `.hpp`) and implementation files (`.cpp`) is only discussed in Section 9.2 of P. J. Deitel and H. M. Deitel (2017), so, if you prefer, you may combine the interface and implementation in one file for the first two assignments, thus stick to the style as shown in Chapters 1 to 8 of P. J. Deitel and H. M. Deitel (2017).

An exception is sub-assignment 1.1, where the focus is to develop insight in the difference between combining and splitting interface and implementation: splitting is compulsory.

From Assignment 3 onwards you *must* separate the interface from implementation.

1.1 File structure and build process

In this sub-assignment you practice compiling and linking C++ files from the command line (i.e., a terminal window). This gives you more insight in the 'magic' that happens when you press the 'Compile and run' button of your IDE.

See Section 5.5 of the RELbot Manual for details on how to compile from command line.

1.1.1 Compile and run simple divide program

Put the code of Listing 1.1 in a file.

Use the command line to compile and run the file.

Show in your report that the program works and explain the outcome. You do not need to hand in the source code for this subsubassignment.

1.1.2 Separate into two source files

Split the code into two `.cpp` files; One containing the `main()` function; the other containing the `divide()` function. Add header files if necessary. Use the command line to compile and link. Make sure you use separate commands for the compiling process and the linking process. Also, make sure that the program runs correctly. You need to hand in the files of this subsubassignment as Assignment 1.1.

Explain what you have put in which file and why (a full listing of each file helps).

Explain what each command-line command (i.e., the compile command and the link command) does and what file(s) it generates.

1.1.3 Compile error

Start from the code of Section 1.1.2. Introduce a compile error. Explain in your report how you introduced the error. Also show and explain what happens when you try to compile and link the source files from the command line. You do not need to hand in any code.

1.1.4 Link error

Start from the code of Section 1.1.2. Introduce a link error. Explain in your report how you introduced the error. Also show and explain what happens when you try to compile and link the source files from the command line. You do not need to hand in any code.

```
#include <iostream>

int divide(const int num, const int den) {
        return num/den;
    }

int main(void) {
        int a = 7;
        int b = 3;

        int c = divide(a, b);
        std::cout << a << "/" << b << "=" << c << std::endl;

        return 0;
    }
```

Listing 1.1: Simple divide program.

1.1.5 Runtime error

Start from the code of Section 1.1.2. Introduce a runtime error, which does not need to be a fatal one (hint: this can be done with just a variable change).

Explain in your report how you introduced the error. Also show and explain what happens when you try to compile and link the source files from the command line. You do not need to hand in any code.

1.2 BMI Calculator

To determine whether a person is overweight or obese, you can use a measure called the Body Mass Index (BMI). The formula for calculating the BMI is:

$$\text{BMI} = \frac{\text{weight in kilograms}}{(\text{height in meters})^2} \tag{1.1}$$

(a) Create a BMI calculator application that reads the user's weight in kilograms and height in meters, and then calculates and displays the user's body mass index. Structure your program well, use functions!

(b) Create a function called evaluateAndPrintBMI that displays the category of the calculated BMI according to the table below. For example, when the calculated BMI is 21, the application should say something like "Your weight is normal.".

(c) The application should display the following information from the Department of Health and Human Services/National Institutes of Health, so the user can evaluate his/her BMI. You should create a separate function called printInfo for this.

BMI VALUES:	
Underweight:	less than 18.5
Normal:	between 18.5 and 24.9
Overweight:	between 25 and 29.9
Obese:	30 or greater

1.3 *Classical only*: Heart Rates

Classical Only
This sub-assignment is *not* for CBL students. Only for those who take the *Classical* version of SDfR.

While exercising, you can use a heart-rate monitor to check that your heart rate stays within a safe range suggested by your trainers and doctors. According to the American Heart Association (AHA), the formula for calculating your maximum heart rate in beats per minute is 220 minus your age in years. Your target heart rate is a range that is 50-85% of that age-specific maximum.

(a) Create a class to store information about humans: the class attributes should include the person's first name, last name and date of birth (consisting of separate attributes for the month, day and year of birth).
(b) The class should have a constructor that receives this data (first name, last name, birthday, birth month, birth year) as parameters. Provide set and get functions where they make sense (for example, it's nice to be able to get a person's first name, but it does not make much sense to set a person's first name separate from the last name). The set function for the date of birth should check that the day (1-31, let's simplify) and month (1-12) are in the correct range.
(c) The class also should include a function getAge that calculates and returns the person's age (in years). If you do not know how to obtain the current date from the computer, you should prompt the user to enter the current month, day and year before calculating the person's age.
(d) Create a function calculateMaximumHeartRate that calculates and returns the person's maximum heart rate and a function calculateTargetHeartRates that calculates and returns the person's minimum and maximum target heart rate.
(e) Write an application that prompts for the person's information and uses your class for human info (instantiate an object). Next, the application should show the full name, date of birth, the person's age (years), maximum heart rate, and target heart rate range. Test your program; for example, test that it validates the dates entered by the user.

1.4 Questions

(a) Given the files from Section 1.1.2, and assuming that we compile and link all file with one single command.
    1. If you forgot to #include a header file in a .cpp file, do you get a compiler error or a linker error? Why?
    2. If you omitted one .cpp file from the compile command (e.g., g++ a.cpp ... instead of g++ a.cpp b.cpp ...), do you get a compiler or a linker error? Why?
(b) Why do you have to #include *<iostream>* in your code?
(c) What is the difference between private and public access specifiers in a C++ class?
(d) Explain the purpose of data hiding.
(e) Give the output of the code shown in Listing 1.2 and explain.

1.5 Submission

Hand in your work to Canvas with two files:

- A *single*-column *.pdf* file with explanations of the design choices you made in the assignments and the answers to the questions of all sub-assignments.
  Call this file <Group number>-SDfR-Assignment1.pdf. For example, for Group 2 of SDfR the file is called 02-SDfR-Assignment1.pdf.
- A *.zip* file which contains your C++ code structured as described in Chapter 5 (Section 5.2) of the RELbot Manual and that all your code files have the standard file header as

```
#include <iostream>

void function();

int a = 20;

int main() {
        int a = 10;
        function();
        std::cout << a;
        return 0;
    }

void function() {
        std::cout << a << std::endl;
    }
```

Listing 1.2: Code for question (e).

described in Chapter 5 (Sections 5.3 and 5.4) of the RELbot Manual (van Oort et al., 2025). Ensure that each subassigment contains a README.txt file in which you describe how to compile and run each program from the command line (including the exact commands that you use).
Remember: Make sure that the TAs can easily compile and run your code.
Call this file <Group number>-SDfR-Assignment1.zip. For example, for Group 8 of SDfR the file is called 08-SDfR-Assignment1.zip.

2 The Maze

The grid of hashes and dots in Figure 2.1 is a two-dimensional 12-by-12 array representation of a maze. In this two-dimensional array, the hashes represent the walls of the maze and the dots represent squares in the possible paths through the maze. The `x` marks the maze entrance. Moves can be made only to a location in the built-in array that contains a dot.

There is a simple algorithm for walking through a maze that guarantees finding the exit (assuming there is an exit). If there is no exit, you will arrive at the starting location again. Place your right hand on the wall. If the maze turns to the right, you follow the wall to the right. As long as you do not remove your hand from the wall, eventually you will arrive at the exit of the maze. There might be a shorter path than the one you have taken, but you are guaranteed to get out of the maze if you follow this algorithm.

Note: For this assignment, you do not need to split up your code into the different subassignments and hand them in separately (as stated in Chapter 5 (Section 5.2) of the RELbot Manual, since both subassignments contribute to the same working maze solving algorithm. Instead, put all your files in a single project called assignment2.

```
# # # # # # # # # # # #
# . . . # . . . . . . #
. . # . # . # # # # . #
# # # . # . . . . # . #
# . . . . # # # . # . x
# # # # . # . # . # . #
# . . # . # . # . # . #
# # . # . # . # . # . #
# . . . . . . . . # . #
# # # # # # . # # # . #
# . . . . . . . . . . #
# # # # # # # # # # # #
```

Figure 2.1: Two-dimensional array representation of maze.

2.1 Arrays

(a) To store the maze fields you can use a two dimensional-array using std::array from the standard library. To use std::array you need to include *<array>*. To see how a such an array is made look at http://en.cppreference.com/w/cpp/container/array. Initialize a maze with the maze in Figure 2.1.

(b) Make a function that prints all the elements of the maze, just as in Figure 2.1.

2.2 Recursion

Remember: you do not need to create a new directory for this subassignment.

(a) Make a function that determines the start position of the maze by searching the array for the starting x. You can use this function later to start your recursive traversal function (next part).

(b) Write a recursive function traverseMaze to walk through the maze. As traverseMaze attempts to locate the exit from the maze, it should place the character x in each square of the path it has visited. The function should display the maze after each move, so the user can watch as the maze is solved. When the maze is solved the program should print a text that the maze is solved and exit the program.

(c) Adapt the recursive function traverseMaze so that a scoped enumerator Direction is used for the direction that it is trying to move to.

2.3 Questions

(a) Explain in your own words what recursion is.
(b) What can be the added benefit of using enumerators over, for example, integers?
(c) When would you use the const keyword?
(d) What is the advantage of the C++-style std::array type in comparison to C-style arrays (e.g. int a[])?
(e) For each of the following code blocks, find the error and explain why it is wrong:

(1)

```
#include "array"
```

(2)

```
if (a = 1) {
    b = 10;
}
```

(3)

```
int b[10] = {};
for (int i = 0; i <= 10; ++i)
    b[i] = 1;
```

(4)

```
const int arraySize = 20;
// ...
arraySize = 10;
```

2.4 Submission

Remember: for the maze assignment, you do not need to split your code into different directories for the two subassignments.

Hand in your work to Canvas with two files:

- A *single*-column *.pdf* file with explanations of the design choices you made in the assignments and the answers to the questions of all sub-assignments.
  Call this file <Group number>-SDfR-Assignment2.pdf. For example, for Group 2 of SDfR the file is called 02-SDfR-Assignment2.pdf.
- A *.zip* file which contains your C++ code structured as described in Chapter 5 (Section 5.2) of the RELbot Manual and that all your code files have the standard file header as described in Chapter 5 (Sections 5.3 and 5.4) of the RELbot Manual (van Oort et al., 2025). Ensure that each subassigment contains a README.txt file in which you describe how to compile and run each program from the command line (including the exact commands that you use).
  Remember: Make sure that the TAs can easily compile and run your code.
  Call this file <Group number>-SDfR-Assignment2.zip. For example, for Group 8 of SDfR the file is called 08-SDfR-Assignment2.zip.

3 Lists and searching

Provided via Canvas (liststrees.zip) are C++ implementations of a linked list and a tree based on the textbook (P. J. Deitel and H. M. Deitel, 2017, Chapter 19). Download it and use it as a starting point for this assignment.

You can use the structure in the provided code as a starting point for your hand in file structure.

3.1 Linked List

(a) Start from the LinkedList starter kit. Use the main.cpp file and include the header file needed to use the linked list List class. Instantiate two empty linked lists (the type of the list elements should be char).

(b) Create a function fillList that takes a string and a char list as argument and fills that list with the chars from the string.

(c) Use fillList to fill the two lists instantiated in the main function with two strings "singlylinkedlist" and "abcdefg". Print the two lists to the screen using their print method.

(d) Extend the List class (so edit List.h) with a function concatenate which takes a List as argument. This function should concatenate the list in the argument to the current list. The argument list needs to be emptied. You may *not* use the insert* and remove* functions from the List class.

(e) In your main function, create two new lists and fill them with "hijklmnop" and "qrstuvw". Use your concatenate method to append these two lists to the list that contains "abcdefg". Print the appended list.

3.2 *Classical only*: Tree

Classical Only
This sub-assignment is *not* for CBL students. Only for those who take the *Classical* version of SDfR.

(a) Start from the Tree starter kit. Use the main.cpp file. Instantiate a tree of integers and fill it with 10 different (unsorted!) predefined integers.

(b) Create a search function in the Tree class that attempts to locate a specified value in a binary search tree object. The search function should take the search key to locate as argument. You can choose between a recursive search implementation (create a search helper function) or an iterative search implementation. If the node containing the search key is found, the function should return a pointer to that node; otherwise, the function should return nullptr.

(c) Extend your main program such that it asks the user to input a number. The program should search your predefined array for the user's number using the search function created previously. The program should output whether the number was found or not.

(d) Create a member function outputTree to display a binary tree object on the screen. The function should output the tree row by row, with the top of the tree at the left of the screen and the bottom of the tree toward the right end of the screen. Each row is output vertically. For example, the binary tree illustrated in Figure 3.1. Use a recursive algorithm for this.

```
            50
        48
            47
    42
        33
27
            20
        17
    13
            11
        6
            1
```

Figure 3.1: Example output of outputTree function.

3.3 Questions

(a) What are the differences between a Linked List, a Stack, a Queue, and a Tree?
(b) Give example usages for these four different data structures.
(c) When do you use the & sign and when do you use the * sign when using pointers?
(d) You define a new pointer variable, but you do not have an address yet to initialise the pointer with. With what value should you initialise that pointer and why?
(e) At the end of (or during) your Linked List program, the Lists' nodes are destroyed, and the program prints the destroyed node elements. Because you instantiated more than one list, you see the order in which the lists are destroyed. Explain this order: what determines the order in which the lists are destroyed, is the order always the same? Note: this is about the destruction of the List objects themselves, not their elements.

3.4 Submission

Hand in your work to Canvas with two files:

- A *single*-column *.pdf* file with explanations of the design choices you made in the assignments and the answers to the questions of all sub-assignments.
  Call this file <Group number>-SDfR-Assignment3.pdf. For example, for Group 2 of SDfR the file is called 02-SDfR-Assignment3.pdf.
- A *.zip* file which contains your C++ code structured as described in Chapter 5 (Section 5.2) of the RELbot Manual and that all your code files have the standard file header as described in Chapter 5 (Sections 5.3 and 5.4) of the RELbot Manual (van Oort et al., 2025). Ensure that each subassigment contains a README.txt file in which you describe how to compile and run each program from the command line (including the exact commands that you use).
  Remember: Make sure that the TAs can easily compile and run your code.
  Call this file <Group number>-SDfR-Assignment3.zip. For example, for Group 8 of SDfR the file is called 08-SDfR-Assignment3.zip.

4 Inheritance

In this assignment, you will learn how to use inheritance to improve your code and your development experience drastically. This is partly because you will need to think about the structure of your inheritance beforehand, and visualize it using a class hierarchy diagram.

A few suggestions for making diagrams on a computer:

- Draw.io (has both an online version on draw.io and an offline version (Help|Get Desktop in the online version)).
- UMLet (https://www.umlet.com/), specific for UML diagrams.
- Dia (http://wiki.gnome.org/Apps/Dia), diagram editor with a rather classical interface.
- Your own favorite application.

Both Draw.io and UMLet have integration into VS Code; check the VS Code Extensions list. Diagrams drawn on paper are accepted as long as they are readable and complete.

4.1 Package Inheritance Hierarchy

Package-delivery services, such as DHL® and UPS®, offer a number of different shipping options, each with specific associated costs and other attributes.

For example, there are the TwoDayPackage and OvernightPackage package options. The cost of shipping a package depend on the type of package. The cost of a TwoDayPackage is 2.50 euro per kg, with a flat fee of € 5. The cost of an OvernightPackage is equal to a TwoDayPackage but with the additional cost of the weight squared times 1.10 €/kg^2.

Your task is to write an object-oriented program that creates a list of a few packages with different shipping options (can be hard-coded). This list of packages should then be passed to a function printCosts that loops over this list and prints the cost of shipping for each package. The cost for each package must obtained by calling the package's member function calculateCost.

- Create a class hierarchy (inheritance) diagram to represent the various types of packages. Use Package as the base class name. Include the diagram in your report AND as the original file/photo in your .zip. At least the following information should be stored for each package:
  - Name and address of the sender.
  - Name and address of the receiver.
  - Package's weight.
- Develop the software. Note that the function printCosts does not need to know about the different types of packages (i.e., if a new package type would be added, the printCosts function should not need to be modified at all!).

4.2 *Classical only*: Customer hierarchy

Classical Only

This sub-assignment is *not* for CBL students. Only for those who take the *Classical* version of SDfR.

Note: This sub-assignment extends the work you did in Section 4.1. Make sure though that you hand in the full programs of each sub-assignment separately. This means that you should copy the code from Section 4.1 into a new directory and continue upon the copied code for Section 4.1. See Chapter 5 (Section 5.2) of the RELbot Manual on structuring C++ projects.

At the package-delivery company they noticed that they often send packages to/from the same customers. Therefore, they want the software to be updated such that the customer information is no longer stored in the package objects, but in a separate list of Customer objects.

The package delivery has two types of customers:

- A PrivateCustomer, which has a Name and Address, and
- A BusinessCustomer, which has a Company Name, Address, and a Contact Person.

Now instead of the name and address of the sender and receiver, each package should store a C++-reference to the sender and receiver Customer.

- Extend the class hierarchy diagram from Section 4.1 accordingly (i.e. create a second one). Do not forget to include the relation between the package classes and customer classes. Include the diagram in your report AND as the original file/photo in your .zip.
- Develop the software. Include a list of customers. There should be at least three customers of each type. Update the package list to get proper references to the customer objects. Make sure that there is at least one customer of each type that is *not* linked to any package (i.e., a customer who is not sending/receiving anything at the moment). Also make sure that there is at least one Business Customer who is involved in multiple packages.
- Make a function printCustomerInfo that for each customer (also those who are not linked to any package) prints the stored data (as listed above). This function does not need to know about the different types of customers (i.e., if a new customer type would be added, the printCustomerInfo function should not need to be modified at all!).
- Now the package-delivery company wants to send a Christmas card to all Contact Persons of only the Business Customers that are currently sending or receiving a package. Write a function, printChristmasCardContactPersons that prints a list of *only the contact person names* (thus, no other information of the Customer) of the according Business Customers. Make sure that each contact person is printed only once (hint: create a temporary list to keep track of which contact persons were already printed).
  Note: It is not allowed to include an identifier in the objects that tells which type of Customer it is (i.e., if the Customer is a BusinessCustomer). Instead, you need to find out, at run-time which type of Customer object you are dealing with (dynamic_cast, typeid).

4.3 Questions

(a) Explain why inheritance is used in the package delivery assignment.
(b) In the first assignment you have explained the difference between private and public access specifiers. What is the difference between those and the protected access specifier?
(c) Explain composition and inheritance in common words, using the topic "computer" (as a start: what classes/objects are related to a computer?)
(d) The compiler “inserts” calls to destructors in your code. Where are these calls to destructors inserted? (in other words, when are they called during program execution?)
(e) What is a virtual destructor, when is it necessary and why?

4.4 Submission

Important: do not forget to hand in your class hierarchy diagrams by adding them to your PDF file AND adding the original image file to the .zip file.

Hand in your work to Canvas with two files:

- A *single*-column *.pdf* file with explanations of the design choices you made in the assignments and the answers to the questions of all sub-assignments.
  Call this file <Group number>-SDfR-Assignment4.pdf. For example, for Group 2 of SDfR the file is called 02-SDfR-Assignment4.pdf.

- A *.zip* file which contains your C++ code structured as described in Chapter 5 (Section 5.2) of the RELbot Manual and that all your code files have the standard file header as described in Chapter 5 (Sections 5.3 and 5.4) of the RELbot Manual (van Oort et al., 2025). Ensure that each subassigment contains a `README.txt` file in which you describe how to compile and run each program from the command line (including the exact commands that you use).
  Remember: Make sure that the TAs can easily compile and run your code.
  Call this file `<Group number>-SDfR-Assignment4.zip`. For example, for Group 8 of SDfR the file is called 08-SDfR-Assignment4.zip.

5 Sequence controller

In this assignment you get familiar with the RELbot, and with ROS2. You furthermore do a first step in developing software for a sequence controller nodes for the RELbot, given a skeleton to work in.

You must use Linux, because ROS 2 runs on Linux. We use ROS 2 version Humble that runs best on Ubuntu 22.04, *Jammy Jellyfish*. Both are the *Long-Term Stable* versions that run with our firmware on the RELbot. If you have not yet done so, install Linux, version Ubuntu 22.04, on your laptop. We provide installation instructions to install Ubuntu 22.04 and ROS2 Humble as *Virtual Machine* on your laptop in Chapter 4 of the RELbot Manual (van Oort et al., 2025).

5.1 RELbot controller architecture

In this subassignment, you will construct diagrams to think ahead for the realisation of the robot software in the final assignment (Chapter 6). Doing so, gives you a flavour of working along approaches of *model-based software development*.

Details on the RELbot hardware and software are in Chapters 13 and 15 of the RELbot Manual (van Oort et al., 2025).

5.1.1 Block diagram

Construct a high-level block diagram of the structure of the RELbot (Cyber-Physical System) for which you are developing the control software. Cover the complete CPS / Robotic system.

- Detailing should be to the level of ROS 2 nodes and topics or the control stack; similar detail is expected for the other parts. So, distinguish essential functionality and draw that as separate blocks.
- The blocks should be detailed on the level of ROS2 nodes and topics.

A few suggestions for making diagrams on a computer:

- Draw.io (has both an online version on draw.io and an offline version (Help|Get Desktop in the online version)).
- UMLet (https://www.umlet.com/), specific for UML diagrams.
- Dia (http://wiki.gnome.org/Apps/Dia), diagram editor with a rather classical interface.
- Your own favorite application.

Both Draw.io and UMLet have integration into VS Code; check the VS Code Extensions list. Diagrams drawn on paper are accepted as long as they are readable and complete.

5.1.2 Details of each block

For each block, specify at least the following aspects:

- For each module/node:
  - Name
  - Description of its function
- For each topic/communication channel:
  - Signal Name
  - Data type
  - For signals representing physical quantities: unit

To avoid cluttered diagrams, or too small text sizes in the diagrams, you can use colour coding or abbreviations (adhere to conventions and standards). While doing so, do provide a legend or a list explaining the abbreviations.

5.2 Getting familiar with ROS2

In this assignment you get familiar with ROS2 and build some projects with it. See the RELbot Manual (van Oort et al., 2025), Chapter 4 on how to get access to ROS2 and the required software, and Chapter 6 on developing software using ROS2.

An overview of all tutorials of ROS 2 is on its website: https://docs.ros.org/en/humble/Tutorials.html.

Do at least these tutorials:

1. Using turtlesim, ros2 and rqt
2. Understanding nodes
3. Understanding topics
4. Launching nodes
5. Creating a workspace
6. Understanding parameters
7. Using parameters in a class (C++)

Tips

- More about the concept of nodes is on their ROS 2 documentation Concepts page: https://docs.ros.org/en/jazzy/Concepts/Basic/About-Nodes.html
- And for topics on https://docs.ros.org/en/jazzy/Concepts/Basic/About-Topics.html
- The *Concepts* pages might be useful to get to know more about the concepts of ROS2.

You do not have to hand in anything for this subassignment.

5.3 Sequence Controller

In this part, you will implement the algorithm for a sequence controller, whereby a skeleton of a ROS2 node is given. See Section 15.7 of the RELbot Manual (**re**) for details how to use it. The code of this skeleton ROS2 node is available on Canvas as ROS2 package including a launch file.

For the RELbot we provide a simulator, and visualisation of the robot pose (position and orientation) via the demo program *Turtlesim*. See the RQt-graph in Figure 5.1.

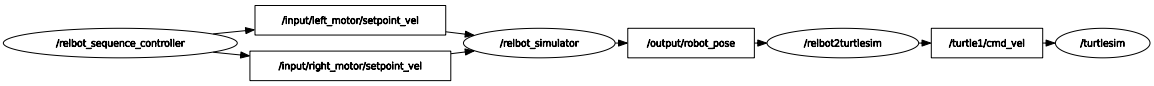

Figure 5.1: RQt graph of the provided nodes. The Sequence Controller is called *seq_controller*. Applying the accompanying launch file results in graph shown here.

5.3.1 Get familiar / Test provided code

Let the RELbot (uh, turtle) move in its plane by publishing different velocities to the wheels from the command line, using the ros2 topic pub <...> command.

You do not have to hand in anything for this subassignment.

5.3.2 Sequence controller

Extend the given Sequence Controller such that it lets the RELbot (turtle) move along different paths:

1. In a straight line.
2. In a circle.
3. First in a straight line and then make a left turn of 90°.
4. Along a square-like path, with at the corners bends with a reasonable but clearly visible radius.

Put in your report the characteristics of the paths (length of straight parts; diameter of the curves), and explain why the turtlesim screendumps show that your RELbot is moving correctly.

Tips

- To generate a stream of setpoints, you could use the *wall-clock time* via the function call get_clock()
- Submit as code for this subassignment *only* your updated / enhanced version of the Sequence Controller package.
- Adhere to the structure of ROS2 projects, as explained in Chapter 6 of the RELbot Manual (van Oort et al., 2025). Besides this being ROS2 professional practice, it allows easy submission and grading.

5.3.3 *Classical only*: Interactive sequencing

Classical Only
This sub-assignment is *not* for PPD students. Only for those who take the *Classical* version of SDfR.

Create a feature using ROS2 parameters to let the RELbot change path while driving.

Show as evidence a screendump of the arbitrary path you made, including the commands you gave.

5.4 Questions

Five questions to be answered arrive soon.

5.5 Submission

Important: For the code of your Sequence Controller, *only* hand in your Sequence Controller package, of course including readme and launch file.

Keep the structure of the files as is, so adhering to the standard structure of files for ROS2 projects, see also Chapter 6 of the RELbot Manual.

That chapter also contains tips on what must be in the README file, how to prepare your ROS2 project for submission, and what TAs do to grade your code.

Hand in your work to Canvas with two files:

- A *single*-column *.pdf* file with explanations of the design choices you made in the assignments and the answers to the questions of all sub-assignments.
  Call this file <Group number>-SDfR-Assignment5.pdf. For example, for Group 2 of SDfR the file is called 02-SDfR-Assignment5.pdf.
- A *.zip* file which contains your C++ code structured as described in Chapter 5 (Section 5.2) of the RELbot Manual and that all your code files have the standard file header as described in Chapter 5 (Sections 5.3 and 5.4) of the RELbot Manual (van Oort et al., 2025). Ensure that each subassigment contains a README.txt file in which you describe how to compile and run each program from the command line (including the exact commands that you use).
  Remember: Make sure that the TAs can easily compile and run your code.
  Call this file <Group number>-SDfR-Assignment5.zip. For example, for Group 8 of SDfR the file is called 08-SDfR-Assignment5.zip.
- Precisely adhere to the structure of ROS2 projects while adding the ROS2 C++ code to the zip file. See Chapter 6 of the RELbot Manual (van Oort et al., 2025).

6 Final Assignment SDfR: Controlling the RELbot

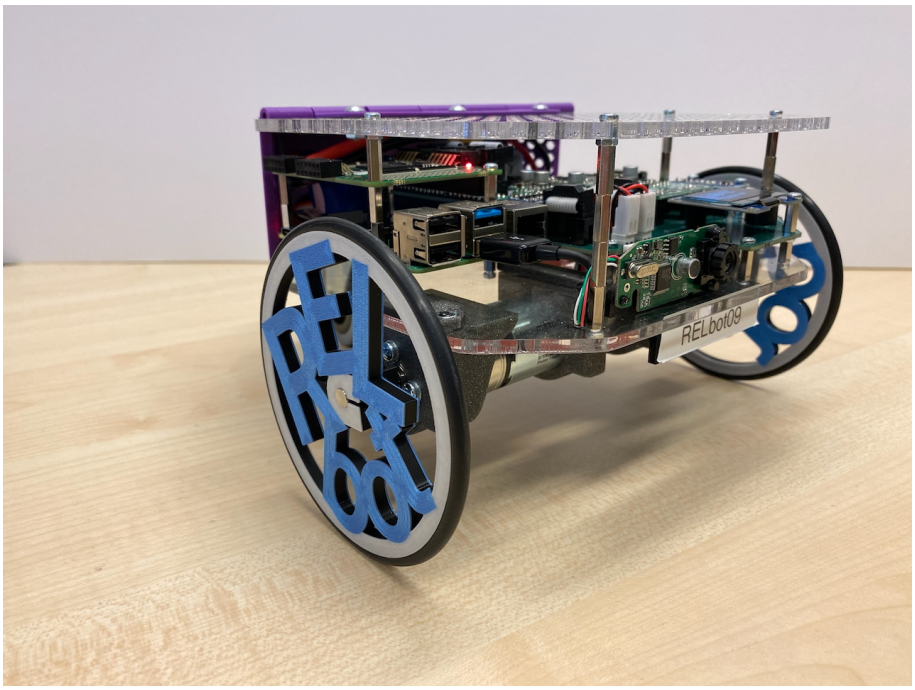

Figure 6.1: RELbot v1.1.

6.0 Introduction

In this final assignment you develop controllers for the RELbot (version 1.1, Figure 6.1). The focus is on the supervisory and sequence controllers (see Figure 6.2, all developed using ROS 2. The hard real-time parts, like loop controller and interfacing to the motors and sensors, are provided. However, some tuning might be needed.

All code runs on the Raspberry Pi of the RELbot, the SRT parts on Linux (Ubuntu 22.04) with ROS2 *Humble* and the FRT parts run on EVL / Xenomai 4.

Notes

- We use on the RELbot indeed Ubuntu 22.04 and ROS2 *Humble*, so *not* the newest Long-Time Stable version. However, ROS2 Humble is the newest version that is compatible with the RELbot firmware. This is the reason why all ROS2 development for the RELbot uses ROS2 Humble on Ubuntu 22.04.
- The new webcam is mounted upside down, because space to place the PCB was only available at the right side of the middle, and the sensor itself is on the left side of that PCB. This implies that you have to flip the image. This is supported in our *updated* cam2image ROS2 package, using an openCV function.

6.0.1 RELbot in the Lab

The first few parts of this assignment is using ROS2 on your own laptop, to further prepare for software development on the RELbot itself. The larger part of this assignment consist of developing code for and on the RELbot's Raspberry Pi. Several RELbots are available in the RaM Lab (Carré 3434) at the same time.

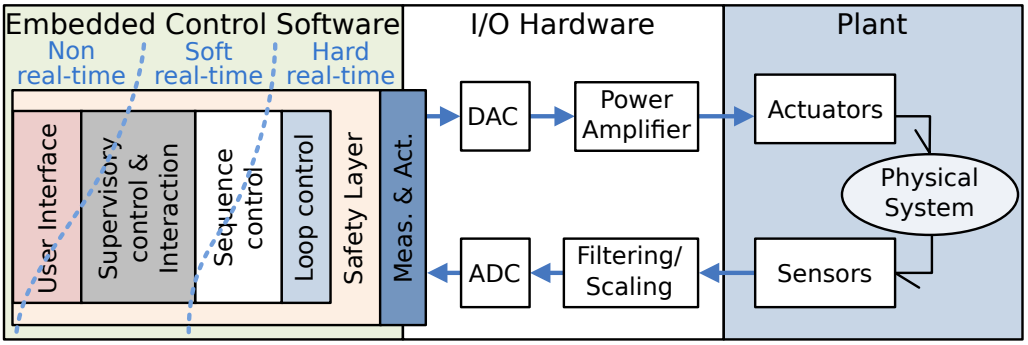

Figure 6.2: Embedded Systems software architecture.

For this assignment, you do *not* need to use the same RELbot during all time slots. The RELbots are identical (except a small variation in some physical parameters) and you should have a copy of you files on your local computer anyway, so switching to a different RELbot and its Raspberry Pi should not be a problem.

Each group is allowed to use 2 two-lecture-hour time slots per week on a Raspberry Pi / RELbot, during the weeks this assignment is scheduled. Timeslots are available on Canvas for claiming.

You *must* subscribe to a time slot in Canvas before you are allowed to use it.

See the RELbot Manual (van Oort et al., 2025), Chapter 8 for access to the RELbots and its Raspberry Pi; Chapters 13 – 16 for details of the RELbot and its software; and Chapters 9 – 10 for the XRF2 framework and its data logger.

The time you have at a setup is limited, therefore it is vital to be well-prepared.

Structure your ROS2 projects as specified in Chapter 6 of the RELbot Manual and adhere to the submission instructions in the same chapter. Just like for Assignment 5.

6.0.2 Planning and intermediate feedback

For each subassignment, except Assignment 6.1, you have the opportunity to let a TA check your work. During that check, you must demonstrate to the TA that your code works and how you validated that it worked. The TA can then provide you with tips. Use this check as a dedicated feedback moment.

Note: Having checked Assignment 6.3 *is* mandatory before working on the physical RELbot (Assignment 6.4). This because of lab equipment (RELbot !!) safety.

To support you keeping on track, a suggested planning for the final assignment is given in Table 6.1.

Note that week 13 and week 16 are is shorter, due to late publication of this Assignment Set and UT being closed on Good Friday respectively.

Week	Assignment	Topic
13*	6.1, 6.2	Cam2Image, Bright Object
14	6.3	Loop Controller in simulation
15	6.4, 6.5	Control the real RELbot
16*	6.6	Slack time; Extras

Table 6.1: Suggested planning.

6.0.3 Tips for preparing your submission

See Section 6.7 for tips, reminders, and rules for the report and zip you have to hand in for this assignment

6.1 Getting more familiar with ROS2

ROS2 has good tutorials on their website, to learn concepts and skills in a step-by-step manner: http://docs.ros.org/en/jazzy/Tutorials.html.

In addition to Assignment 5.2, do the following tutorials:

1. Creating a package
2. Writing a simple publisher and subscriber (C++)

You do not have to hand in anything for this assignment.

6.2 Detect a coloured object

This subassignment is about interpreting the image from the webcam, to later steer the RELbot. As we start in the development environment using the RELbot simulator, use the webcam of your laptop to mimic the effect the movements of the RELbot model have on the view seen by its webcam. This way, this work can *all* be done using the simulator on your laptop.

Due to Multipass VM or WSL Linux not being able to access the webcam directly, the webcam stream must be sent over a UDP channel. For this, we provide on Canvas a videoserver script and an updated image_tools ROS2 package, see Chapter 7 of the RELbot Manual (van Oort et al., 2025).

6.2.1 Test the updated image_tools

Test the updated image_tools ROS2 package (node cam2image_vm2ros) on your laptop to capture webcam footage, publish to a channel and view the channel using the standard showimage node of the standard image_tools ROS2 package.

The standard showimage node can be started using the following command in a separate shell, whereby the X-server needs explicitly enabled when running on a Mac / Multipass VM (argument –X at the ssh command):

```
ros2 run image_tools showimage
```

Remarks and Tips

- Details about the updated cam2image package are in Chapter 7 of the RELbot Manual.
- Make a launchfile for this test. This makes inspecting your work while grading doable.
- Put the design decisions you made, evidence of your work (some results) in the report. Checking Section 6.0.3 might be beneficial.

6.2.2 Image processing: detect the coloured object

Develop a ROS2 node that detects the coloured (green) object visible in the moving camera output of the RELbot simulator. Let it furthermore interpret the image to find the x, y position where the camera looks at. This output will later be sent to the Sequence Controller developed in Section 6.3.

Make a test bed by connecting this node to the cam2image_vm2ros node and thus using your laptop's webcam image stream. Show the outputs of the Node Under Design (*NUD*) on the terminal, or more elegantly, connect the output to *Turtlesim*, maybe re-using your setup of Section 5.3.

Remarks and Tips

- Use the input of the light-position indicator, /moving_camera_output of the RELbot simulator (just as later in the real case, where the RELbot moves during operation and thus let the webcam see moving images of the world).
- Let the NUD produce its outputs in a pace comparable to the pace of its input stream from the webcam, so 30 FPS.
- Make a launchfile for this test. This makes inspecting your work while grading doable.
- Put the design decisions you made and evidence that the system works (some results) in the report.
- To prepare for submission, see the tips in Section 6.0.3.

6.3 Sequence Controller

Extend your Sequence Controller node made in Section 5.3 such that the simulated RELbot follows the coloured (green) object that you move around in front of your webcam. Use as input the results produced by the node developed in Section 6.2.2.

Compute appropriate setpoints to let the simulated RELbot move to have the coloured object in the centre of the view of the webcam.

Remember to keep the system modular (i.e., keep the node(s) easily re-usable; thus not too specialised).

In order to make this work, you need to steer the RELbot pose (position and orientation), $\mathbf{x}_{\text{RELbot}} = [x_{\text{RELbot}} \quad \theta_{z,\text{RELbot}}]^T$, towards the (absolute) position of the light $\mathbf{x}_{\text{light}}$ such that the error, $\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}$, becomes zero.

A convenient controller choice is a first order controller, where the rate of change of the RELbot orientation, $\dot{\mathbf{x}}_{\text{RELbot}}$, is proportional to the error, i.e.,

$$\dot{\mathbf{x}}_{\text{RELbot}} = \begin{bmatrix} \dot{x}_{\text{RELbot}} \\ \dot{\theta}_{z,\text{RELbot}} \end{bmatrix} = \frac{1}{\tau} \cdot \left(\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}\right) = \frac{1}{\tau} \cdot \begin{bmatrix} x_{\text{light}} - x_{\text{RELbot}} \\ \theta_{z,\text{light}} - \theta_{z,\text{RELbot}} \end{bmatrix}; \quad \mathbf{x}_{\text{RELbot}} = \int \dot{\mathbf{x}}_{\text{RELbot}} \, \mathrm{d}t, \tag{6.1}$$

where τ is some time constant (a good start would be $\tau = 1$s).

Develop the system described above. For the integration (second equation of (6.1)) you can use the Forward Euler integration method.

Remarks and Tips

- The position of the coloured spot in the camera image is precisely $\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}$.
- Moving in the x direction means zooming in or zooming out.
- Moving around the z axis (changing θ_z) means horizontally moving the moving-camera cut-out over the webcam image.
- The only way to influence the camera orientation is by sending a setpoint $\mathbf{x}_{\text{set}}$ to the simulator node. Fortunately, the RELbot can track its setpoint rather well, so we can assume that $\mathbf{x}_{\text{RELbot}} = \mathbf{x}_{\text{set}}$, resulting in

$$\dot{\mathbf{x}}_{\text{set}} = \begin{bmatrix} \dot{x}_{\text{set}} \\ \dot{\theta}_{z,\text{set}} \end{bmatrix} = \frac{1}{\tau} \cdot \left(\mathbf{x}_{\text{light}} - \mathbf{x}_{\text{RELbot}}\right) = \frac{1}{\tau} \cdot \begin{bmatrix} x_{\text{light}} - x_{\text{RELbot}} \\ \theta_{z,\text{light}} - \theta_{z,\text{RELbot}} \end{bmatrix}; \quad \mathbf{x}_{\text{set}} = \int \dot{\mathbf{x}}_{\text{set}} \, \mathrm{d}t. \tag{6.2}$$

- As the RELbot is a differential-driven wheeled robot, its kinematic structure is relevant. Measures of the RELbot you need here: Width of the robot: 20 cm; Diameter of the wheels: 10 cm. See Chapter 13 and 15 of the RELbot Manual.
- Let the node produce its outputs in a pace that is needed for the node where it is connected to: either the simulator or the real robot. Good values lie between 33 and 10 ms; 1 ms could be too short as this node runs on ROS2.

- Make a launchfile for this test. This makes inspecting your work while grading doable.
- Put the design decisions you made and evidence that the system works (some results) in the report. So, show that the simulated RELbot can track the chosen object.
- To prepare for submission, see the tips in Section 6.0.3.

Sign off and results

You must show the result of Assignment 6.3 to one of the TAs in order to receive feedback. You can use this as a dedicated feedback moment.

Note: at this point you are ready to move from simulation to controlling the real robot.

Therefore, signing off is mandatory.

6.4 Controlling the physical RELbot

Finally, using the real RELbot !!

Now it is time to implement the entire controller-code stack as tested in simulation in the previous assignment (Section 6.3) on the real RELbot.

We do this in a step-by-step manner to have easy work while testing.

Chapters 13 and 8 of the RELbot Manual describe the provided RELbot hardware and how to access the Raspberry Pi on the RELbot being in the Lab.

Note that **signing off (Section 6.3) is required** before you are allowed to run code on the physical RELbot.

6.4.1 Controller-code stack on the Raspberry Pi of the RELbot

The first step is to test the ROS2 code of the entire controller-code stack on the Raspberry Pi of the RELbot, without activating the motors. So, using the RELbot simulator instead of the real motors. This to check whether migration of the code to the embedded processor does work, and behaves similar to running on your own laptop. In other words, is the Raspberry Pi processor strong enough to run the code, and computes the results in the same way as on the laptop (a Raspberry Pi has a different CPU architecture compared to a laptop, in general).

Migrate the entire controller-code stack *together with* the RELbot simulator, so the work of Section 6.3 to the Raspberry Pi of the RELbot.

Run the ROS 2 nodes, that is, run the simulation, to show that the system still works as expected.

The standard showimage node can be started using the following command in a separate shell, X-server enabled (argument –X at the ssh command) running on the Raspberry Pi started from your laptop, of course:

```
ros2 run image_tools showimage
```

Remarks and Tips

- rqt is not installed on the RELbot's Raspberry Pi (it can*not* easily be installed). Fortunately, showimage *is* available.
- To show graphical results, you must use *X forwarding*. Enable the X-server while starting the ssh connection, using ssh -X <user>@<RELbot-IPv4 address>, see Section 4.8 of the RELbot Manual.
- The RELbot's Raspberry Pi does not have a graphical environment, so it is quite obvious that you cannot see contents of graphical windows directly on the Raspberry Pi.

6.4.2 Basic test of the provided loop controller

The next step is to check whether the provided loop controller works as expected. Just to be sure, and to test this part separately. Furthermore, you might discover possible mistakes in setpoints, which could damage the robot when this step would be skipped.

Construct a ROS 2 node that provides setpoints to the loop controller that is provided. Let these two parts run together, and ***leave the RELbot on its stand!!!!***

Run some basic tests like driving each motor separately (the other motor does not turn), driving both: a straight ride, and some turns.

Evaluate these tests: does the system behave as expected?

You may show evidence in the report by providing a link to a short movie showing some rotation of the wheels of the RELbot.

Remarks and Tips

- The provided loop controller runs on the FRT part, on the Xenomai core of the Raspberry Pi using the XRF2 framework. See Chapter 9 of the RELbot Manual on how to use the Xenomai part.
- Chapter 9 of the RELbot Manual is, is about developing code on the FRT part of the Raspberry Pi. For only running existing code, use the part *Buuild and Run* at the end of that chapter.
- The used project name is Xenomai with RelBotDemo as class implementing the functionality.
- The Loop Controller code is generated by 20-sim and the controller submodel is of course named LoopController().
- Put the design decisions you made and evidence that the system works (some results) in the report. So, show that the simulated RELbot can track the chosen object.
- To prepare for submission, see the tips in Section 6.0.3.

6.4.3 Questions

1. Does the simulated system perform on the Raspberry Pi as well as on your own laptop? Why (not)?
2. Did you have to make changes to the controller-code stack while migrating it from your laptop to the Raspberry Pi? If so, why was it necessary? Could you have avoided this in the previous subassignments?
3. Did the setpoints you provide in Section 6.4.2 match with those you made in your controller stack? If not, what did you need to change, that is, what did you *not* take into account while simulating?

6.5 Final Integration

Finally, integration time!!

6.5.1 Integrate all and final demonstration

Combine your controller-code stack and the provided loop controller, to really control the RELbot. The goal is to let the code work immediately right (*"Doing right first time"*), to optimally profit of all the hard work in the previous steps, namely carefully developing and testing the different parts that are combined in this step.

Given the results of the previous subassignment, you might need to tweak your own controller-code stack.

Let the RELbot follow the coloured ball.

You could do the same or comparable experiments as you did before when testing your controller-stack code with the RELbot simulator.

Evaluate these tests: does the system behave as expected?

Show evidence in the report by providing a link to a short movie showing your RELbot move as result of following the coloured ball.

Remarks and Tips

- A few simple moves are enough.
- Take care to let the RELbot *not* to move too fast!! Moving the coloured ball gently helps in letting the RELbot make some movements.
- ***Challenging the RELbot to its limits is fun, but it is more important to keep the RELbot intact!***
- Zooming might not work, as the webcams have autobalancing of light intensity. If this is the case, driving forward / backward as reaction on moving the coloured ball is not necessary to show.
- Put the design decisions you made and evidence that the system works (some results) in the report. So, show that the simulated RELbot can track the chosen object.
- To prepare for submission, see the tips in Section 6.0.3..

6.5.2 Questions

1. Did the final integration (Section 6.5) work immediately, that is *first-time right*?
   If not, what did you need to change to your software while integrating all parts? In hindsight, how could you have avoided the need of changing in the last, so integration phase? (Note: mismatch between simulation and reality is often the case, so seeing differences is not at all a fault from your side!)

6.6 Extra Functionality (Bonus Assignment)

In this bonus assignment you can add functionality to the already working setup. Some ideas:

- Start-up procedure
- Advanced behaviour in the case no bright light is seen.
- Add a supervisory controller to, for instance, switch on / off certain controllers/nodes, or change mode/parameter values of these nodes. See Figure 6.2 and slideset 5-SwStructuring-ROS, slides 8 – 14.
- Safety layer: RELbot stops after seeing an obstacle (slide 5-14).
- Extra safety: stop the movement of the whole robot if one of the degrees of freedom does not function as expected.
- Your own ideas...
- Put the design decisions you made and evidence that the system works (some results) in the report. So, show that the simulated RELbot can track the chosen object.
- To prepare for submission, see the tips in Section 6.0.3.

Describe what extra functionality you want to make and how it will influence the behaviour of the whole system. Explain how you implemented the extra functionality. Motivate the design choices. Show and discuss test results.

Note: As with all assignments, the focus is on developing *good, well-structured solutions*. A very cool feature which is just 'hacked' into the system will not give you points.

6.7 Submission

While preparing for submission, remind the following:

- Reused code from previous assignments and ROS2 tutorials does not need to be motivated (again): a reference to your earlier work (or to ROS2 tutorials) is enough.
- **Important**: Hand in *only* code you made yourself. So *not* and *never* add code we provided.
- Explain in the report *what* you tested, *why* you tested that and *how* you tested it.
- Make sure that your programs / ROS2 nodes can be easily compiled and run by the TAs by providing proper README.txt file(s), see Chapter 6 of the RELbot Manual.
- **Note**: Store results of subassignment separately, that is, prepare it for submission (see Chapter 6 of the RELbot Manual) and set it apart such that it can be graded separately. Obviously, this implies that for the ROS2 code, you make a separate ROS2 package or put the node in some relevant package.

Hand in your work to Canvas with two files:

- A *single*-column *.pdf* file with explanations of the design choices you made in the assignments and the answers to the questions of all sub-assignments.
  Call this file <Group number>-SDfR-Assignment6.pdf. For example, for Group 2 of SDfR the file is called 02-SDfR-Assignment6.pdf.
- A *.zip* file which contains your C++ code structured as described in Chapter 5 (Section 5.2) of the RELbot Manual and that all your code files have the standard file header as described in Chapter 5 (Sections 5.3 and 5.4) of the RELbot Manual (van Oort et al., 2025). Ensure that each subassigment contains a README.txt file in which you describe how to compile and run each program from the command line (including the exact commands that you use).
  Remember: Make sure that the TAs can easily compile and run your code.
  Call this file <Group number>-SDfR-Assignment6.zip. For example, for Group 8 of SDfR the file is called 08-SDfR-Assignment6.zip.
- Precisely adhere to the structure of ROS2 projects while adding the ROS2 C++ code to the zip file. See Chapter 6 of the RELbot Manual (van Oort et al., 2025).

A Change Log

- **version 2025.1**, 4 February 2026
  - First version of the Assignment Manual of Academic Year 2025/2026, for both Programming 2 and Software Development for Robotics.

B Course-Specific Framing for PPD Projects

As stated in the PPD manual, the course-specific framing is:

- SDfR
  Describe the CPS architecture of the robotic system.
- ASDfR
  Describe the CPS architecture of the robotic system. Include also the real-time aspects.

Further details are presented here. First the common things, then specifics for ASDfR, and also in case your group has both SDfR and ASDfR students on board.

B.1 Describe the CPS architecture of your robotic system – both SDfR and ASDfR

It is on presenting the architecture of the software, interfacing to hardware, and hardware of your robotic system / embedded control system / cyber-physical system.

From the point of view of developing software for robots, the terms embedded control system and cyber-physical system are often used interchangeably.

Provide at least the following

- Block diagram of the 'top level' of the system (high-level block diagram), comparable to ROS2 nodes, and use a similar detail for non-ROS2 parts.
- Next to this block diagram, specify of each software part, next to its name in the diagram:
  - Description of its function.
  - Where it runs, that is on which processor (type), if applicable.
  - Sample time (if applicable).
  - Interfaces do not need to be specified but may be mentioned, as details of the connected communication channels also indicate what is produced as output, expected / received as input.
- Specify for each communication 'channel', next to its name in the diagram:
  - Type of channel (ROS, data, network package, electrical signal).
  - Data type, size of data.
  - Physical quantity and unit, in case physical signals are communicated.
  - Sample time (if applicable).
- Specify for the non-software parts:
  - Description of its function.
  - What domain it is in (electrical, mechanical...).
  - If available and useful, more domain-specific details.
  - Interfaces do not need to be specified, unless the communication channels are not described separately (for instance user interface, interface to environment). Then describe that interface.

B.2 Specifics for ASDfR

In addition to the information stated for SDfR, add:

- Augment the block diagram to indicate timing regime: Non / Soft / Firm-Hard real-time / Physics / etc. You may indicate those timing regimes by drawing (dotted) lines to separate those areas.
- What timing support each software block needs to have: SRT, F/HRT, or NonRT.
- What QoS and timing behaviour each communication channel needs to have. You may use ROS2 terminology, if applicable and useful.
- How connections between SRT and F/HRT parts are realised.

B.3 Combined SDfR and ASDfR

In case your CBL group has both SDfR and ASDfR students, you may specify both parts as a combination.

B.4 Hints on producing diagrams

A few suggestions for making diagrams on a computer:

- Draw.io (has both an online version on draw.io and an offline version (Help|Get Desktop in the online version)).
- UMLet (https://www.umlet.com/), specific for UML diagrams.
- Dia (http://wiki.gnome.org/Apps/Dia), diagram editor with a rather classical interface.
- Your own favorite application.

Both Draw.io and UMLet have integration into VS Code; check the VS Code Extensions list. Diagrams drawn on paper are accepted as long as they are readable and complete.

C Useful resources

C.1 Books

There are many bad C++ books, in the sense of for example containing incorrect information or teaching very bad programming style and design. Use this list to filter: https://stackoverflow.com/questions/388242/the-definitive-c-book-guide-and-list

C.2 Online

- C++ language FAQ: https://isocpp.org/faq
- C++ standard library: http://www.cplusplus.com/reference
- C++ tutorial: http://www.cplusplus.com/doc/tutorial
- C++ Core Guidelines: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- "The Cherno" on Youtube: https://www.youtube.com/watch?v=18c3MTX0PK0&list=PLlrATfBNZ98dudnM48yfGUldqGD0S4FFb

Bibliography

Deitel, Paul J. and Harvey M. Deitel (2017). *C++ How to Program*. 10th. Prentice Hall. ISBN: 978-1-292-15334-6, or 978-0-134-44823-7.

Student Charter (2020). *Student Charter at University of Twente*. URL: https://www.utwente.nl/en/ces/sacc/regulations/charter.

van Oort, G., L.B.L. Lenders, and Broenink (2025). *RELbot Software and Usage Manual (v2025.n)*. Manual.